

REVAL: Real-Time Edge Vibration Analysis for Home Standby Generator Fault Detection

Project Type: Research Project

Alex Ouellette*, Derek Crocker, Taewoong Yoon, Logan Pelkey

School of Electrical and Computer Engineering, Purdue University

West Lafayette, IN

ouellea@purdue.edu, crockerd@purdue.edu, yoon312@purdue.edu, lepelkey@purdue.edu

*Team Leader

Abstract—Home standby generators are critical residential infrastructure but lack real-time health monitoring in typical installations. This paper presents REVAL, an end-to-end IoT system that performs vibration-based fault classification entirely on an ESP32 microcontroller and streams decision-level results to a cloud-free local backend. A jitter-free 1 kHz MPU-6050 acquisition pipeline (hardware-timer ISR pinned to Core 1, with Wi-Fi and lwIP isolated on Core 0) feeds a 44-feature ESP-DSP front end and a TensorFlow Lite Micro MLP that emits an 8-class decision approximately twice per second; results, telemetry, and fault-transition alerts are multiplexed over HTTP to a FastAPI/WebSocket backend and a React dashboard. The full pipeline fits within approximately 58 kB DSP+inference working set, leaving more than 260 kB of on-chip SRAM for the network stack. On a 3,282-window holdout drawn from a 12,323-window corpus collected from a 22 kW home standby generator, the deployed 44–64–32–8 MLP achieves 90.19% healthy-vs-fault binary accuracy, 69.07% 8-class accuracy, and 0.7357 macro-F1 at 12–15 ms on-device inference latency, demonstrating that residential-grade predictive maintenance is deliverable on a sub-\$35 MCU-class platform without cloud dependency.

Index Terms—vibration analysis, fault detection, edge inference, ESP32, TinyML, TensorFlow Lite Micro, IoT, predictive maintenance

I. INTRODUCTION

Rotating-machinery failure is a dominant contributor to industrial downtime and maintenance cost. Maintenance expenditure on rotating equipment can represent between 15% and 60% of total manufacturing cost [1], and predictive maintenance programs have been shown to reduce maintenance cost by 25–30% and eliminate 70–75% of breakdowns, yielding a ten-fold return on investment [2]. Home standby generators, a class of small rotating machinery that had a global market value of roughly USD 8.5 billion in 2024 [3], share the dominant failure modes of industrial equipment (mechanical looseness, air-intake restrictions, load imbalances, and, as future work, combustion faults) yet remain almost entirely unmonitored in residential installations.

Vibration-based condition monitoring is a mature technique in industrial predictive maintenance [4]. Characteristic spectral signatures appear in an accelerometer signal long before catastrophic failure, enabling early detection along the P–F curve [5]. Mechanical looseness shifts energy into sub-

harmonic and low-frequency bands depending on the coupling involved; intake restrictions and load imbalances shift the dominant-frequency distribution; combustion misfires (not targeted in this work) produce impulsive transients. These signals are accessible to a low-cost triaxial MEMS accelerometer sampled at kilohertz rates. However, deploying such a system in a residential setting demands that inference run locally: cloud pipelines introduce latency, depend on always-on Internet connectivity, and raise privacy concerns around sensing inside the home.

This work presents **REVAL (Real-time Edge Vibration Analysis)**, a complete edge-to-dashboard system that closes this gap for residential generators. REVAL performs all sampling, digital signal processing, feature extraction, and machine-learning inference on an Adafruit ESP32 Feather V2 and publishes only decision-level metadata over Wi-Fi. The system distinguishes five healthy operating conditions (0%, 25%, 50%, 75%, and 100% of rated load) from three mechanical fault classes (air-filter restriction, loose mounting bolt, and loose panels) for a total of eight classes, and was validated against data collected on an instrumented 22 kW home standby generator.

The primary contributions of this paper are:

- 1) A **complete, reproducible edge-ML pipeline** for generator fault detection that fits in a single ESP32 firmware image, including jitter-free 1 kHz tri-axial acquisition, 44-feature DSP, a TFLite Micro FP32 MLP, and a three-stream network publisher.
- 2) A **contract-based subsystem architecture** consisting of fixed C-struct hand-offs that decouple the HAL, DSP, ML, and networking subsystems, paired with a drop-in *recorded-sensor* back-end that lets the full production pipeline be exercised against a flash-resident dataset in place of live MPU-6050 samples.
- 3) An **experimental evaluation** over a ~ 6.3 M-sample dataset collected from a real 22 kW generator, including a systematic comparison of three MLP depths, FP32 versus INT8 quantization, and compact 44-feature versus extended 74-feature representations.

The remainder of this paper is organized as follows. Sec-

tion II reviews related work on edge-deployed vibration fault detection. Section III describes the system architecture. Section IV details the hardware and sampling pipeline. Section V covers the DSP front end. Section VI describes the training-data collection campaign. Section VII presents the ML runtime, model selection, and on-device integration. Section VIII describes the network publisher and visualization stack. Section IX presents the experimental results. Section X discusses limitations, Section XI describes team member contributions, and Section XII concludes with directions for future work.

II. RELATED WORK

Vibration-based fault detection for rotating machinery is well-studied; recent effort has shifted from server-side analytics toward edge deployment.

Olokede and Ashigwuike [6] trained XGBoost, MLP, and hybrid predictors on two months of vibration, oil-pressure, and temperature data from a 500 kVA diesel generator, reporting a failure-prediction accuracy of 1.000 with an MLP and 0.994 with XGBoost. Their result confirms that dramatic generator faults are visible in vibration data alone, but the study targeted a single large industrial unit (500 kVA versus the 1–50 kVA typical of residential machines [3]) and relied on server-side processing. REVAL explicitly targets the residential generator size class and places all inference on the MCU.

Lee *et al.* [7] reported 99.5% classification accuracy across normal operation and three fault conditions (unbalance, misalignment, shaft rubbing) using four accelerometers and a modified VGG19 CNN with Global Average Pooling. Replacing the fully connected head with GAP reduced parameters approximately seven-fold. Even so, the approach assumes GPU-class hardware for training and inference and is unsuitable for MCU deployment.

Two recent studies demonstrate that lightweight networks can run on microcontroller-class hardware. Asutkar *et al.* [8] introduced a TinyML transfer-learning framework for bearing-fault diagnosis that generalizes across machine domains from as few as 25 labeled samples per class, and demonstrated deployment on an ESP32. Huang *et al.* [9] ported a MobileNet-based model to an STM32F405 (Cortex-M4, 192 kB SRAM), achieving 98% accuracy at 136 ms inference latency while reducing transmitted data volume to 0.1% of the raw stream. Arciniegas *et al.* [10] reported 96.5% accuracy at 25 ms inference on an ESP32-S3 using an MPU-6050 with INT8 quantization under Edge Impulse. Of these three, [10] is the closest analogue to REVAL (the same MPU-6050 / ESP32-class combination) and provides a direct point of comparison: REVAL’s deployed model attains a lower 8-class accuracy (69.07%) on a taxonomy that mixes five healthy load levels with three mechanical-fault classes, and roughly halves the inference latency (12–15 ms vs. 25 ms). On the binary healthy-vs-fault framing (closer to the binary anomaly task in [10]), REVAL achieves 90.19% accuracy.

REVAL is closest in spirit to the MCU-deployed work of [8]–[10] and departs from it along three deliberate axes. We acknowledge a roughly 6 pp absolute gap to [10] on

the binary framing (90.19% vs. 96.5%) and trade some of that absolute accuracy for a more diverse fault taxonomy and for a deployable software stack. First, REVAL targets residential generator-specific fault modes rather than generic bearing or motor faults; to our knowledge, no prior MCU-deployed vibration-classification work has targeted residential-scale standby generators specifically. Second, it uses engineered frequency-domain features rather than raw-signal input; this choice dramatically reduces model complexity (the deployed network has 5,224 parameters versus 10^4 – 10^6 typical of CNN-based approaches), at the cost of a hand-specified feature pipeline. Third, it couples the on-device classifier to a complete network-and-visualization layer, enabling the demonstration of an end-to-end IoT pipeline rather than an isolated inference benchmark.

III. SYSTEM ARCHITECTURE

Fig. 1 summarizes the REVAL data path. Raw samples flow from the MPU-6050 through the HAL layer into a double-buffered windowing stage, a 1024-point DSP/feature-extraction stage, a TFLite Micro inference engine, and a Wi-Fi publisher that feeds a FastAPI backend and a browser dashboard.

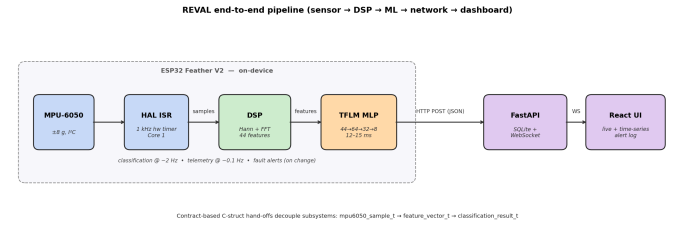


Fig. 1. End-to-end REVAL data path from sensor to dashboard.

The firmware exposes two compile-time build modes (**Data Collection** and **Production**) and two sensor back-ends (**real MPU-6050** and **recorded sensor**), yielding four concrete PlatformIO environments (Table I). The *recorded-sensor* back-end compiles a pre-captured CSV into a flash-resident C array whose playback is indistinguishable from the real driver from the perspective of the rest of the pipeline; this allowed subsystems (DSP, ML, networking, UI) to be developed and exercised against a flash-resident dataset rather than requiring live generator access, removing contention over the single physical unit.

TABLE I
BUILD ENVIRONMENTS AND FLASH FOOTPRINT

Environment	Sensor	Pipeline	Flash
data_collection	MPU-6050	Sample→CSV/UART	~180 kB
data_coll_rec.	Flash	Sample→CSV/UART	~200 kB
production	MPU-6050	Sample→DSP→ML→Net	~350 kB
production_rec.	Flash	Sample→DSP→ML→Net	~370 kB

All inter-subsystem hand-offs occur through fixed C struct contracts in a shared integration header (HAL→buffer manager, DSP→ML, and ML→network). This decoupled the

subsystems: each producer could be stubbed with a CSV file or a constant struct while the consumer was iterated in isolation.

IV. HARDWARE AND SAMPLING

Platform. The target is the Adafruit ESP32 Feather V2, which pairs a dual-core 240 MHz Xtensa LX6 SoC with 320 kB of on-chip SRAM, 8 MB of flash, and an integrated 2.4 GHz Wi-Fi radio. The vibration sensor is an InvenSense MPU-6050 six-axis IMU configured for ± 8 g full-scale accelerometer range with the gyroscope disabled to reduce I²C bus traffic. Communication uses I²C Fast Mode (400 kHz) over GPIO22/GPIO20. The MPU-6050’s digital low-pass filter (DLPF_CFG = 0) is left at its default 260 Hz cutoff, which attenuates content above ~ 260 Hz before digitization; the spectral-band features in Section V are defined over the full 0–500 Hz range, but the upper half of that range reflects DLPF-filtered content.

Jitter-free sampling. Early prototypes using FreeRTOS software timers exhibited ± 1 ms of per-sample jitter, which is unacceptable for spectral analysis: a bin width of ~ 1 Hz at a 1 kHz sample rate requires sample periods stable to well under 1% of the nominal 1000 μ s. The final firmware replaces software timers with a hardware timer that generates an interrupt every 1000 μ s. The ISR gives a FreeRTOS binary semaphore to a high-priority sampling task pinned to Core 1. Core 0 is reserved for the Wi-Fi and lwIP stacks. Keeping Wi-Fi off the sampling core eliminates the tail-latency spikes that would otherwise occur during RF activity and back-off. The sampling task reads the three accelerometer registers over I²C and pushes a `mpu6050_sample_t` (x, y, z in milli-g, plus a 64-bit timestamp) into the current half of the double buffer. Measured post-fix, the per-sample period held within a few microseconds of 1000 μ s under concurrent Wi-Fi activity, which was a prerequisite for any of the downstream spectral-band features to be meaningful.

Calibration. A two-second boot-time stationary average (~ 2000 samples) provides the per-axis DC bias that is subtracted from each subsequent reading, yielding zero-centered `int32` milli-g regardless of sensor orientation.

Recorded-sensor back-end. To enable off-site development the HAL exposes a second sensor driver that reads from a C array in DROM. A Python conversion utility transforms a labeled CSV collected during Data Collection mode into a C header that compiles into flash (~ 12 kB per second of data). The recorded driver honors the same 1 kHz callback cadence and wraps cyclically; the rest of the pipeline cannot distinguish it from the I²C driver. This mechanism was essential for bringing the DSP, ML, and network subsystems online without blocking on generator access.

V. SIGNAL PROCESSING PIPELINE

The DSP front end converts a continuous stream of per-sample tri-axial data into a fixed-dimension feature vector, at a rate matched to the expected fault timescale.

Windowing. A 1024-sample sliding window (1.024 s of data at 1 kHz) with 50% overlap is used. Overlap is implemented with a ping-pong double buffer (two 1024×3 `int32`

arrays, 24 kB total) so that sample acquisition and feature extraction proceed concurrently. The 50% overlap yields a nominal decision rate of ~ 1.95 Hz. The window length was chosen as a power of two to admit a radix-2 FFT, and is long enough to resolve the dominant rotational frequencies of a 3600 rpm generator (60 Hz fundamental) while short enough to localize transient impacts.

FFT and windowing function. The per-window mean is subtracted from each axis prior to feature extraction, so the time-domain statistics and FFT input are both zero-mean; this removes any residual DC/gravity component that survived boot-time calibration. Before the FFT, each axis is then multiplied by a 1024-point Hann window (precomputed at boot). The FFT is performed via `dsps_fft2r_fc32()` from Espressif’s `esp-dsp` library, which exploits the Xtensa LX6 DSP instructions and completes a 1024-point complex-FFT well within the 500 ms inter-window budget. The Nyquist frequency is 500 Hz, which comfortably covers the mechanical fundamentals and low-order harmonics of interest for a four-pole 60 Hz generator.

Feature vector. Each window produces a 44-dimensional feature vector organized as three axis blocks plus a small cross-axis block (Table II). Per axis, five time-domain features (RMS, variance, peak, kurtosis, crest factor) capture amplitude and impulsiveness, and nine frequency-domain features (dominant-frequency-in-Hz plus eight equal-width spectral-band energies spanning 0–500 Hz, 62.5 Hz per bin) capture the distribution of spectral energy. Two cross-axis features (magnitude-RMS and magnitude-peak of the DC-removed vibration vector $\sqrt{\hat{x}^2 + \hat{y}^2 + \hat{z}^2}$) provide orientation-invariant indicators for installations where sensor mounting orientation cannot be tightly controlled.

TABLE II
DEPLOYED 44-FEATURE LAYOUT

Block	Features	N	Purpose
Per-axis time	RMS, var., peak, kurt., crest	$5 \times 3 = 15$	Amplitude, impulse
Per-axis freq.	Dom. freq., 8 band energies	$9 \times 3 = 27$	Spectral dist.
Cross-axis	Mag. RMS, mag. peak	2	Orient.-invariant
Total		44	

Memory budget. The DSP working set consumes approximately 42 kB of SRAM, dominated by the 24 kB ping-pong acquisition buffer with the remainder split between FFT scratch, the Hann coefficient table, and the magnitude spectrum. This leaves headroom for the TFLite Micro tensor arena (16 kB) and for the Wi-Fi and lwIP stacks on Core 0.

Feature-count ablation (44 vs. 74). A second, extended feature definition was explored that retains the same five per-axis time-domain features and widens the frequency-domain block to 18 features per axis (dominant frequency, 16 finer-grained bands of 31.25 Hz each, and a spectral centroid), together with five cross-axis features (magnitude-RMS, magnitude-peak, and three inter-axis RMS ratios), for a total of 74 features. An empirical comparison on the same

holdout did not show a meaningful accuracy gain over the 44-feature version, so the 44-feature pipeline is the shipped configuration and the 74-feature pipeline remains a research track.

Python/C++ parity. A reference implementation of the feature pipeline was written in NumPy/SciPy and used for offline training. A dedicated parity test feeds identical windows through the ESP32 (running in Data Collection mode with an additional feature-dump hook) and through the Python reference, and asserts element-wise agreement within a float32-rounding tolerance. Passing this parity test was a prerequisite for integrating any model trained offline: without it, a model trained in Python could exhibit silent accuracy loss on-device. This parity discipline is what allowed the ML and DSP subsystems to be developed on different machines and then composed without integration surprises, and it is the reason the FP32 TFLite holdout accuracy matches the Keras source to floating-point precision.

VI. DATA COLLECTION

Training data was acquired directly from a 22 kW home standby generator instrumented with the MPU-6050 taped to the alternator housing. The dataset used for the results reported here was collected across three sessions (11 Mar 2026, 21 Apr 2026, and 22 Apr 2026) after the mounting was confirmed and the wiring harness was finalized; earlier exploratory captures at different sample rates and mounting configurations were excluded from training to avoid introducing rig-specific distribution shift.

During collection, firmware was flashed in `data_collection` mode, which streams a seven-field CSV row (sequence number, timestamp, tri-axial acceleration in milli-g, label ID, and label string) per sample over UART at 921,600 baud (chosen for a 15% headroom above the 50 kB/s steady-state data rate). A host-side Python logger captures the stream to per-session CSV files, and supports in-run label switching via `CMD:LABEL:<class>` to retain provenance across fault injections within a single session.

Fault injection. Faults were injected by physical manipulation of the generator: the air filter was partially occluded with a heavy piece of cardboard (air-filter restriction); one of the mounting bolts securing the alternator to its base pad was loosened five turns (loose mounting bolt); one of the side access panels as well as the hood was left unfastened and free to rattle (loose panels); and healthy baselines were captured at each of the five electrical load points using a resistive load bank. Two of the ten classes proposed at project kickoff (**unbalanced electric load** and **spark-plug misfire**) were not collected within the project timeline and are absent from the deployed classifier, reducing the class count from the ten proposed to eight actually trained.

In aggregate the raw dataset comprises ~ 6.31 M tri-axial samples across 77 labeled CSV files, yielding 12,323 overlapping feature windows after the 1024-sample / 50%-overlap windowing pipeline. Class balance is uneven: the five healthy-load classes have $\sim 1,000$ windows each, while the

TABLE III
DATA COLLECTION CAMPAIGN

Class	Windows
healthy_0pct	1,002
healthy_25pct	998
healthy_50pct	998
healthy_75pct	999
healthy_100pct	999
airfilter_restriction	1,870
loose_mounting_bolt	2,694
loose_panels	2,763
Total	12,323

three fault classes range from 1,870 (air-filter restriction) to 2,763 (loose panels), so the fault classes are the numeric majority. Section VII-A describes the inverse-frequency class-weighting used to prevent the classifier from over-fitting the more populous fault classes at the expense of the healthy load points.

Deployed-model training corpus. The model weights reported in Sections VII–IX were trained against the full 77-file / 12,323-window corpus summarized in Table III, with a 19-file / 3,282-window held-out test partition produced by a file-disjoint split manifest.

VII. MACHINE LEARNING

A. Model Candidates and Training

The classification problem is a flat 8-class problem over the Table III labels. The target constraints are: a fixed 44-float input, an 8-way softmax output, an inference latency well under the 500 ms inter-window budget, a model payload that fits alongside ~ 42 kB of DSP buffers and the Wi-Fi and lwIP stacks on an ESP32 with 320 kB of SRAM, and a tensor arena on the order of 16 kB. These rule out deep CNNs and favor a small multilayer perceptron (MLP).

Three MLP depths were trained and compared: `mlp_32` (one hidden layer of 32 units), `mlp_64` (64 units), and `mlp_64_32` (64 units, then 32 units). All networks use ReLU activations, a softmax output over 8 classes, the Adam optimizer, a batch size of 32, and up to 100 epochs with early stopping on validation loss (patience = 12). Features were standardized with a `StandardScaler` fit on the training split; the mean and scale vectors were exported alongside the model weights. Training, hyperparameter selection, and the final retrain used a fixed RNG seed of 42.

Hyperparameter search proceeded in two stages: a coarse 12-trial grid varying L2 weight (0 , 1×10^{-4} , 1×10^{-3}), dropout (0 , 0.1), and use of class weights (on/off) at a fixed learning rate of 0.001 , followed by a 16-trial refined grid centered on the coarse winner that additionally varied learning rate (5×10^{-4} , 1×10^{-3}) against L2, dropout, and class weights as before. The selected configuration (used for all reported results) was learning rate 1×10^{-3} , $L2 = 1 \times 10^{-4}$, no dropout, with inverse-frequency class weighting enabled. This configuration was retrained on the file-disjoint train/test partition described in Section VI: 58 files (9,041 windows) for

training and 19 files (3,282 windows) held out, with every class represented in the holdout. File-disjoint splitting is required because the 50% window overlap means a random window-level split would leak sample-level content across the train/test boundary.

TABLE IV
HOLDOUT PERFORMANCE ACROSS MLP CANDIDATES. SIZE COLUMNS REPORT .TFLITE PAYLOAD BYTES. BEST PER-COLUMN VALUES IN BOLD; MLP_64_32 IS THE DEPLOYED MODEL.

Model	Par.	Size (B)		FP32			INT8	
		FP32	INT8	8-cl.	F1	Bin.	8-cl.	Bin.
mlp_32	1704	8556	4768	.702	.745	.884	.701	.883
mlp_64	3400	15372	7360	.681	.724	.852	.676	.849
mlp_64_32	5224	23112	10576	.691	.736	.902	.691	.900

B. Holdout Results

Table IV reports multiclass and binary (healthy vs. any-fault) metrics on the holdout set for all three architectures and for both their FP32 and INT8 TFLite exports.

The two-hidden-layer mlp_64_32 obtained the best binary healthy-vs-fault accuracy (90.19%) and the best macro-F1 (0.7357), and was selected for deployment on that basis. Notably, the smaller mlp_32 achieves a slightly higher 8-class accuracy (70.2%) than mlp_64_32 (69.07%), but loses ~ 1.8 pp on the binary fault-detection metric; model capacity is not the bottleneck for 8-class accuracy on this dataset. The harder decision is *which* fault is present, because vibration signatures vary strongly with generator load and load-dependent fault signatures overlap across classes.

INT8 post-training quantization was also evaluated and produced negligible accuracy loss across all three architectures: on mlp_64_32 the 8-class accuracy moves from 69.07% (FP32) to 69.13% (INT8) and binary accuracy from 90.19% to 90.01%, both within run-to-run noise. The INT8 export halves the .tflite payload (23.1 kB $\rightarrow$ 10.6 kB for mlp_64_32) without measurable accuracy cost, making it a viable path for SRAM-constrained future targets (Section X). For the current ESP32 prototype we deployed FP32 because the SRAM headroom permits it and FP32 simplifies on-device debugging; the INT8 variant is exported alongside as a drop-in replacement for the planned Cortex-M4 port.

C. Naive-Baseline Comparison

Edge-ML is a constrained optimization problem: workstation accuracy is not a sufficient deployment criterion. To characterize where the deployed MLP sits relative to classical alternatives, we trained three baselines on the same 44-feature holdout: logistic regression, an RBF-kernel SVM, and a random forest with 500 trees. Table V compares accuracy and model size side-by-side.

The MLP is competitive on accuracy and dominant on size relative to the kernel and ensemble baselines. The SVM carries roughly $10\times$ more model components than the MLP

TABLE V
NAIVE BASELINES ON THE SAME 44-FEATURE HOLDOUT. “SIZE” USES THE NATURAL UNIT FOR EACH MODEL: TRAINABLE PARAMETERS FOR THE MLP AND LOGISTIC REGRESSION, SUPPORT VECTORS FOR THE SVM, AND TOTAL DECISION NODES ACROSS THE FOREST FOR RF.

Model	Size	8-cl.	Macro-F1	Bin.
Logistic Regression	360 params	0.715	0.779	0.885
SVM (RBF kernel)	52,932 SVs	0.673	0.726	0.892
Random Forest (500)	248,700 nodes	0.618	0.697	0.928
mlp_64_32 (deployed)	5,224 params	0.691	0.736	0.902

(52,932 support vectors at 44 floats each, ≈ 9.3 MB of stored vectors before counting the kernel-evaluation cost) and loses on both 8-class (0.673 vs. 0.691) and binary accuracy (0.892 vs. 0.902). The random forest carries roughly $48\times$ more components ($\sim 248,700$ decision nodes, ~ 10 MB at conservative node-encoding sizes) and is dramatically worse on 8-class accuracy (0.618 vs. 0.691) despite winning binary detection by 2.6 pp (0.928 vs. 0.902).

The random-forest binary advantage is misleading on inspection. The forest collapses fault-class recall to 0.205 on loose-mounting-bolt and 0.224 on loose-panels (versus 0.329 and 0.486 for the MLP), trading per-fault discrimination for inflated binary accuracy by hedging heavily toward the healthy classes. Its *healthy_75pct* precision drops to 0.605, the same dumping-ground pattern discussed in Section IX-A but more severe. The MLP keeps its fault recall higher across all three fault classes, which is the operationally relevant behavior.

Logistic regression is the one baseline that beats the MLP on holdout 8-class accuracy (0.715 vs. 0.691) and macro-F1 (0.779 vs. 0.736), at the smallest size in the table (8 classes $\times$ 44 features + 8 biases = 360 parameters). On binary detection it loses to the MLP by 1.7 pp (0.885 vs. 0.902). It is a viable deployment alternative we did not pursue in this iteration; revisiting it on the Cortex-M4 port is straightforward and is flagged in Section XII. For the current ESP32 deployment the MLP was selected because TFLite Micro provides a turnkey integration path (direct .tflite round-trip, esp-nn optimized FullyConnected kernels, INT8 PTQ with ≤ 0.5 pp loss), none of which are available for a hand-written logistic regression without rebuilding the inference and quantization toolchain.

D. On-Device Runtime

The deployed firmware uses TensorFlow Lite for Micro-controllers (tflite::MicroInterpreter) as the inference runtime, chosen for its stability, the direct round-trip from the .tflite file produced by the training pipeline, and support for the small kernel set we actually need. A MicroMutableOpResolver is initialized with only four operators (FullyConnected, Softmax, Reshape, Relu), keeping the resolver static and compact. A 16 kB static tensor arena is reserved at compile time. The arena fit the two-layer 44-64-32-8 model with headroom; the actual arena bytes reported by arena_used_bytes() are logged at boot for telemetry. The build is configured with CONFIG_NN_OPTIMIZED=y,

so the FullyConnected and Softmax kernels delegate to espn’s hand-tuned Xtensa LX6 implementations rather than to the reference TFLM kernels; combined with the esp-dsp FFT, this is what makes the 12–15 ms per-window inference budget achievable on a 240 MHz core.

Feature normalization is applied inside the inference function, using per-feature mean and scale arrays exported alongside the model weights at training time. This keeps the ML↔DSP contract clean: the DSP layer emits raw features and the ML layer owns the normalization. This also means that swapping in a new trained model requires only replacing the embedded model artifact and its accompanying scaler arrays.

Inference proceeds as (1) standardize 44 floats, (2) memcpy into the TFLM input tensor, (3) invoke the interpreter, (4) memcpy the 8-class softmax out, (5) compute the argmax and confidence, (6) stamp the window ID, inference timestamp, and duration into the result struct, and (7) hand the struct to the network client. Observed per-window inference duration on the 240 MHz ESP32 was in the 12–15 ms range, comfortably below the ~500 ms inter-window budget and well below the 136 ms MobileNet benchmark reported in [9].

VIII. NETWORK AND VISUALIZATION

The on-device publisher exposes three distinct message channels multiplexed over a single HTTP POST endpoint, each with a small fixed JSON schema. A FreeRTOS queue (s_message_queue, depth 32) decouples the inference path from the HTTP sender; inference producers never block on networking.

TABLE VI
PUBLISHED NETWORK STREAMS

Stream	Cadence	Payload
Class.	~2 Hz	window_id, inference_ms, fault class/conf/probs[8], ISO timestamp
Telem.	~0.1 Hz	Uptime, CPU temp., heap, overruns, inference count/avg., RSSI, sensor flag
Alert	On change	Type (FAULT_DETECTED/CLEARED), prev/current state, confidence, count

Serialization. Payloads use hand-rolled sprintf JSON in ~1 kB bounded buffers, avoiding a cJSON dependency. Device IDs are derived from the Wi-Fi MAC, so multi-unit deployments are self-disambiguating.

Transport and backend. The device posts to http://<host>:8000/api/telemetry/ingest (default 10.0.0.144). The backend is a FastAPI service that validates, persists to a SQLite history table, and broadcasts each message to subscribed browser clients over WebSockets. A React dashboard (Vite, React 18) renders live fault readout, confidence time-series, telemetry, and an alert log. A schema-identical Python test publisher produces the same payload format, which decoupled UI development from the availability of working firmware or a working generator.

The publisher drops messages when the FreeRTOS queue fills or HTTP POSTs fail; transport resilience is discussed further in Section X.

IX. EXPERIMENTAL RESULTS

A. Classification Accuracy

On the 3,282-window test partition (drawn from 19 of the 77 source files in Table III under the file-disjoint split described in Section VII-A), the deployed mlp_64_32 FP32 TFLite model matched its Keras source to floating-point precision in both multiclass accuracy (69.07%) and macro-F1 (0.7357), confirming that the offline-to-device export introduced no measurable regression. The binary healthy-vs-fault framing yielded 90.19% accuracy. Fig. 2 shows the per-class confusion matrix on the holdout set.

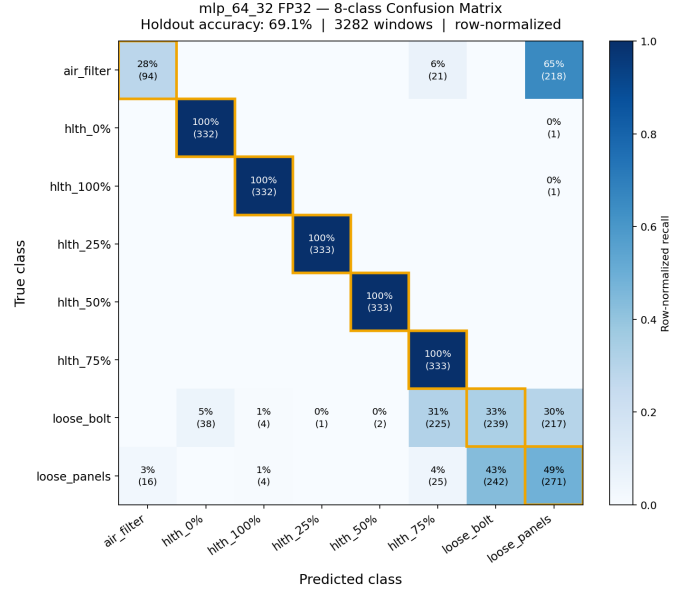


Fig. 2. 8 × 8 confusion matrix on the 3,282-window holdout set (rows = true class, columns = predicted class).

Per-class precision, recall, and F1 on the holdout are summarized in Table VII; the same numbers are visualized in the confusion matrix of Fig. 2.

TABLE VII
PER-CLASS HOLDOUT PERFORMANCE (FP32 MLP_64_32)

Class	Prec.	Rec.	F1	Sup.
healthy_0pct	0.897	0.997	0.945	333
healthy_25pct	0.997	1.000	0.999	333
healthy_50pct	0.994	1.000	0.997	333
healthy_75pct	0.551	1.000	0.711	333
healthy_100pct	0.976	0.997	0.987	333
airfilter_restrict.	0.855	0.282	0.424	333
loose_mount._bolt	0.497	0.329	0.396	726
loose_panels	0.383	0.486	0.428	558

Two patterns dominate the per-class numbers. First, the five healthy load levels are essentially solved ($\geq 99.7\%$ recall each), so they account for roughly 5/8 of the headline 8-class accuracy and inflate it relative to the difficulty of the actual fault-disambiguation problem. Second, the three fault classes are where the model struggles: at the per-window level, the

model misses $\sim 72\%$ of true air-filter, $\sim 67\%$ of true mounting-bolt, and $\sim 51\%$ of true loose-panel windows (Table VII). The 90.19% binary accuracy is therefore meaningfully more representative of deployable behavior than the 8-class number. The classification errors concentrate in the three fault classes, where the model frequently distinguishes *that* a fault is present (the 90.19% binary accuracy) but confuses *which* fault. Air-filter restriction is most often misclassified as loose-panels (both share broad-spectrum mid-band content), and loose-mounting-bolt and loose-panels confuse with each other.

The healthy_75pct precision collapse. A specific failure mode worth calling out is that `healthy_75pct` has perfect recall (1.000) but precision of only 0.551, meaning roughly 45% of windows the classifier *labels* as `healthy_75pct` are in fact faulted. The softmax is using `healthy_75pct` as a “dumping ground” for fault windows whose load-conditioned spectral signature happens to resemble the 75% healthy operating point. This is consistent with the broader pattern: vibration signatures change strongly with generator load, a fault at one load level can resemble a different fault (or a healthy state) at another load level, and without an exogenous load signal (e.g. CAN-bus RPM or electrical load) the classifier cannot fully disentangle load from fault. Operationally this means the binary fault detector is the trustworthy output of the system today; the per-fault label should be treated as a hint rather than a decision until the load confounder is resolved (Section XII).

B. Resource Utilization

Table VIII reports the on-device resource budget.

TABLE VIII
ON-DEVICE RESOURCE BUDGET

Resource	Budget	Used
DSP working set	~ 60 kB	~ 42 kB (ping-pong + FFT scratch + feature vec.)
TFLM tensor arena	16 kB	Fits with headroom
Model payload	—	23.1 kB FP32 .tflite
Classification rate	≥ 1 Hz	~ 1.95 Hz
Inference latency	< 500 ms	12–15 ms
I ² C sample period	1000 μ s	1000 μ s (HW timer)
Sampling core	Core 1	Pinned, priority 24

The combined DSP + ML working set consumes roughly 58 kB of SRAM (42 kB DSP + 16 kB tensor arena), leaving more than 260 kB of on-chip SRAM for the Wi-Fi and lwIP stacks, the FreeRTOS scheduler, the HTTP client, and the message queue.

C. Parity and Export Validation

Three parity tests, each run end-to-end, validated the pipeline. First, the feature-extraction parity test (Section V) matched ESP32 and NumPy features element-wise on identical windows. Second, the export-parity audit exercised all three candidate networks through their Keras source, their FP32 TFLite export, and (separately) their INT8 TFLite export, and confirmed that the Keras and FP32-TFLite outputs agreed

to floating-point precision on the entire holdout set. Third, the recorded-sensor back-end was exercised end-to-end by compiling `production_recorded`, flashing the ESP32, and verifying that the classifications produced on-device over Wi-Fi matched the offline Python predictions on the same recorded samples; this confirmed the HAL $\rightarrow$ DSP $\rightarrow$ ML $\rightarrow$ network pipeline runs without regression on the physical target.

D. Live Hardware-in-Loop Demonstration

The full pipeline was demonstrated end-to-end on the running 22 kW generator during the 28 Apr 2026 demo session. The deployed firmware sampled the live MPU-6050, executed on-device DSP and inference, and streamed classifications and fault alerts over Wi-Fi to the FastAPI backend; the React dashboard rendered the results in real time. Two scenarios were captured:

- 1) **Air-filter restriction at 25% load.** A piece of cardboard was placed over the air intake while the generator ran at 25% electrical load. The dashboard transitioned from a healthy classification to `FAULT_DETECTED` within a few inference windows, and an alert message was published on the state-change stream. After the cardboard was removed, the system returned to a healthy classification and emitted the corresponding `CLEARED` alert. The full chain (physical fault injection $\rightarrow$ MPU-6050 $\rightarrow$ on-device DSP and inference $\rightarrow$ Wi-Fi $\rightarrow$ FastAPI $\rightarrow$ React UI) operated continuously throughout the test.
- 2) **Load sweep across operating points.** The generator was swept across the five electrical load points using the resistive load bank. The dashboard tracked the load transitions in real time, with most windows classified into the correct healthy load class. A subset of windows during transitions were misclassified, consistent with the load-conditioned confusion pattern reported in Section IX-A.

Both demonstrations confirm that the on-device inference loop, the Wi-Fi publisher, and the dashboard operate end-to-end at the design cadence (~ 1.95 Hz classifications, ~ 0.1 Hz telemetry) on real generator vibration.

E. Reproducibility

Code, trained models, and the file-disjoint split manifest are available at <https://github.com/dcrock03/60864-REVAL> (commit 4bf76a0); the deployed firmware is the `production` build environment.

F. Limitations of the Evaluation

Five limitations of the current evaluation warrant disclosure. First, the dataset was collected on a single 22 kW generator unit at a single sensor mount point across three sessions; we have not yet characterized how much of the signal is unit- or mount-specific. Across the three collection sessions, the per-class mean tri-axial RMS shows healthy classes drifting under 5% session-to-session and fault classes



Fig. 3. MPU-6050 mounted to the alternator housing of the 22 kW home standby generator. Sensor orientation and location were held fixed across all three collection sessions.

drifting 8–33% (air-filter restriction was the largest outlier at $\sim 33\%$, plausibly because the cardboard occlusion severity was not standardized across sessions). Second, two classes planned at proposal time (unbalanced electric load and spark-plug misfire, Section VI) were not collected, so the shipping classifier covers 8 of the 10 proposed classes. Third, although the train/test split is file-disjoint (no recording appears in both partitions), all 77 files originate from the same unit, so holdout numbers should be read as an upper bound on generalization to *unseen units* rather than to unseen recordings of the same unit. Fourth, the 8-class accuracy of 69.07% is well below safety-critical production thresholds; the 90.19% binary fault-detection metric is the more deployment-relevant figure but still misses roughly one fault in ten. Fifth, although the live demonstration of Section IX-D confirmed end-to-end operation, per-window classifications were not persisted during the demo, so a quantitative on-device confusion matrix across all eight conditions remains future work (Section XII).

X. DISCUSSION

Accuracy versus footprint. Model capacity wasn’t the trade-off we expected it to be. The deployed `mlp_64_32` has 5,224 parameters and beats the 1,704-parameter `mlp_32` by 1.8 pp on binary detection, but loses to it slightly on 8-class accuracy. Going bigger doesn’t help — the 74-feature pipeline with a wider hidden layer didn’t move the holdout numbers, and it would eat into the Wi-Fi stack’s SRAM budget. The bottleneck on this dataset is the data itself: fault signatures overlap across load levels, and no amount of parameters fixes that.

Engineered features versus end-to-end learning. Most edge-ML papers in this space [8]–[10] feed raw vibration into a small CNN and let the network learn its representation. REVAL inverts this: FFT and feature engineering on-device, then a 44-float vector into a trivial MLP. Training is fast (sub-second per epoch on CPU) and the model is small enough to inspect by hand, but the feature set bakes in assumptions about which vibration statistics matter, and faults outside the mechanical/rotational family likely won’t be visible.

INT8 path. INT8 quantization cuts the `.tflite` payload from 23.1 kB to 10.6 kB and costs us 0.3–0.5 pp of accuracy across all three architectures, which is below the noise floor. We didn’t ship INT8 on the ESP32 because we have the SRAM to spare and FP32 is easier to debug, but it’s the obvious choice for the Cortex-M4 port where flash and SRAM are tighter.

Generalization. The single-unit dataset is the biggest gap between REVAL and a product. Our hypothesis is that frequency-domain features will generalize across units better than amplitude features (60 Hz and its harmonics depend on electrical configuration, while RMS and peak depend on mounting), but this is unvalidated. The next step is a transfer-learning setup along the lines of [8]: pre-train on pooled healthy data, fine-tune per-unit on a small fault set.

Transport resilience. The current publisher drops messages on HTTP failure and on queue overflow, relying on the Wi-Fi manager to reassociate (Section VIII). A bounded-back-off retry queue with a short on-flash persistence layer would let the device ride out multi-second backend outages without losing single-event `FAULT_DETECTED / CLEARED` alerts, which (unlike the redundant classification and telemetry streams) are not self-healing.

Edge versus cloud. On-device inference gives bounded latency (12–15 ms vs. home-uplink variability), continued operation when the WAN is down, and a payload that exposes only decision-level metadata. The trade is per-device model weights and an OTA channel for fleet retraining (not built in this iteration). For residential single-unit deployments edge wins; commercial fleet management warrants a hybrid edge-feature/cloud-classifier split.

IoT system trade-offs. Three deployment concerns warrant explicit discussion. *Security:* the publisher uses plain HTTP with no TLS, bearer-token authentication, or payload signing. This is acceptable on a controlled lab LAN but not for residential deployment, where TLS to the local gateway and a per-device shared secret (or mTLS) should be the minimum bar. The decision-level payload is small and infrequent, so TLS overhead on the ESP32 is well within budget. *Protocol choice:* HTTP was chosen for FastAPI compatibility, ease of debugging, and direct browser-to-WebSocket fan-out. An MQTT broker would suit a multi-unit deployment better, with fewer connections per device, retained messages for last-known-state, and QoS levels for the fault-alert stream; it is the natural transport for the next iteration. *Privacy:* on-device inference keeps raw vibration data on-premises; only the 8-way softmax, summary telemetry, and state-change alerts cross

the network. This matters more than transport encryption, because TLS doesn't help if you're already shipping raw sensor traces to the cloud. Provisioning, time synchronization, multi-unit scaling, and power optimization are flagged as future work in Section XII.

XI. TEAM MEMBER CONTRIBUTIONS

Alex Ouellette (team leader) owned the overall system architecture and hardware platform selection. He designed and implemented the jitter-free 1 kHz sampling pipeline, including the hardware-timer ISR, dual-core partitioning, and double-buffered acquisition; built the recorded-sensor backend and the Python CSV-to-header conversion tooling; ran all three on-site data collection sessions (11 Mar, 21 Apr, and 22 Apr 2026); and coordinated integration across the four subsystems. He also authored the inter-subsystem integration contracts and prepared this report.

Derek Crocker built the C++ DSP and networking subsystems. He implemented the FFT-based feature-extraction pipeline against the `esp-dsp` library, wrote the Python/C++ feature-parity validation tooling, and implemented the three-stream HTTP client and Wi-Fi manager. He chose HTTP over MQTT as the transport protocol after coordinating with Logan on dashboard compatibility.

Taewoong Yoon owned all machine-learning work. He trained the three MLP candidates (`mlp_32`, `mlp_64`, `mlp_64_32`) in TensorFlow/Keras, ran the two-stage hyperparameter search, performed FP32 and INT8 TFLite export and holdout evaluation, and integrated TensorFlow Lite for Microcontrollers into the firmware. He also conducted the 74-feature ablation and the naive-baseline comparison.

Logan Pelkey built the backend and visualization stack. He developed the FastAPI server with SQLite persistence and WebSocket fan-out, implemented the React dashboard (Vite, React 18) with the live fault readout, time-series views, telemetry panel, and alert log, and wrote a schema-identical Python test publisher that let UI development proceed without working firmware.

XII. CONCLUSION AND FUTURE WORK

REVAL demonstrates that residential-grade vibration-based fault detection (sampling, DSP, ML inference, and a Wi-Fi-fed dashboard) can be delivered end-to-end on a single ESP32-class microcontroller. The headline result is 90.19% healthy-vs-fault binary accuracy at 12–15 ms per-window latency on a 22 kW generator dataset; the gap to the 69.07% 8-class accuracy is dominated by load-conditioned spectral overlap (Section IX-A), which motivates the first item of future work below. The two-fold INT8 payload reduction at ≤ 0.5 pp accuracy cost is what makes a future Cortex-M4 SKU realistic.

Five directions define our future work:

- 1) **Exogenous load input.** Integrating a generator CAN-bus signal (RPM, electrical load, oil pressure) as a 45th feature is the highest-impact lever for closing the gap

between the 90.19% binary accuracy and the 69.07% 8-class accuracy, since load is the dominant confounder in the current confusion matrix.

- 2) **Quantify the hardware-in-loop validation.** Live operation has been demonstrated qualitatively (Section IX-D), including a real `FAULT_DETECTED` $\rightarrow$ `CLEARED` transition for an air-filter restriction injected at 25% load. The remaining work is a single continuous live session under all eight fault conditions with backend persistence enabled, yielding a per-class confusion matrix drawn from on-device inference rather than from offline TFLite replay.
- 3) **Broaden the fault taxonomy,** beginning with the spark-plug-misfire class deferred from this report, and continuing with bearing faults and thermal precursors that prior work (e.g. [4], [7]) has shown are visible in vibration.
- 4) **Cross-unit generalization,** via a small multi-unit data collection campaign and a transfer-learning protocol modeled on [8].
- 5) **INT8 and CMSIS-NN paths,** targeting a second firmware SKU on a smaller Cortex-M4-class MCU with a BLE rather than Wi-Fi uplink. Because INT8 PTQ already costs ≤ 0.5 pp on this dataset, the Cortex-M4 port can use the existing INT8 export directly without quantization-aware training.

REFERENCES

- [1] M. Romassini, P. C. C. de Aguirre, L. Compassi-Severo, and A. G. Girardi, "A review on vibration monitoring techniques for predictive maintenance of rotating machinery," *Eng.*, vol. 4, no. 3, pp. 1797–1817, Jun. 2023, doi: 10.3390/eng4030102.
- [2] U.S. Department of Energy, Federal Energy Management Program, "Types of maintenance programs," in *Operations & Maintenance Best Practices, Release 3.0*, ch. 5. Washington, D.C.: U.S. DOE, 2010.
- [3] Global Market Insights, "Home standby gensets market size, growth analysis 2025–2034," Global Market Insights Inc., Selbyville, DE, Rep. ID 3308, 2025. [Online]. Available: <https://www.gminsights.com/industry-analysis/home-standby-gensets-market>
- [4] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. John Wiley & Sons, 2011.
- [5] Sensorfy, "What is the P-F curve in maintenance applications?" *Sensorfy Blog*, Oct. 4, 2022.
- [6] O. Olokede and E. Ashigwuike, "Development of a predictive maintenance algorithm for a diesel generator using machine learning," *Int. J. Innovative Sci. Res. Technol.*, vol. 10, no. 3, pp. 1417–1427, 2025, doi: 10.38124/ijisrt/25mar1226.
- [7] S. Lee, H. Yu, H. Yang, I. Song, J. Choi, J. Yang, G. Lim, K.-S. Kim, B. Choi, and J. Kwon, "A study on deep learning application of vibration data and visualization of defects for predictive maintenance," *Appl. Sci.*, vol. 11, no. 4, p. 1564, Feb. 2021, doi: 10.3390/app11041564.
- [8] S. Asutkar, C. Chalke, K. Shivgan, and S. Tallur, "TinyML-enabled edge implementation of transfer learning framework for domain generalization in machine fault diagnosis," *Expert Syst. Appl.*, vol. 213, Part B, art. 119016, Mar. 2023, doi: 10.1016/j.eswa.2022.119016.
- [9] Y. Huang, S. Liang, T. Cui, X. Mu, T. Luo, S. Wang, and G. Wu, "Edge computing and fault diagnosis of rotating machinery based on MobileNet," *Sensors*, vol. 24, no. 16, art. 5156, Aug. 2024, doi: 10.3390/s24165156.
- [10] S. Arciniegas, D. Rivero, J. Piñan, et al., "IoT device for detecting abnormal vibrations in motors using TinyML," *Discover Internet of Things*, vol. 5, art. 41, Apr. 2025, doi: 10.1007/s43926-025-00142-4.